SVX

SystemVerilog Extension Library

Applications Catalog

Core Library Applications

Version 2.1.3 beta

November 2026

Mark Glasser

Copyright 2026 Mark Glasser. Licensed under the Apache License, Version 2.0.

Contents

1 Sparse Memory **2**
- 1.1 Parameters . . . 3
- 1.2 Types . . . 3
- 1.3 Class: `mem` . . . 4
- 1.4 Class: `mem_bounded` . . . 5

2 Memory Map **6**
- 2.1 Space Types . . . 7
- 2.2 Class: `mem_space` (Abstract Base) . . . 7
- 2.3 Classes: `mem_region` and `mem_view` . . . 8
- 2.4 Class: `mem_register` . . . 8
- 2.5 Classes: `mem_memory` and `mem_field` . . . 9

3 Priority Queue **10**
- 3.1 Class: `pri_queue` . . . 10

1. Sparse Memory

The `mem` application provides a simulation-friendly model of a large address space without requiring backing storage for every possible location. Memory is allocated lazily: pages and blocks are created on demand as data is written, so only those locations that actually receive writes consume simulator memory. A sparse representation makes it practical to model a 32-bit (or wider) address space in a verification environment without pre-allocating four gigabytes of simulator memory.

Address Decomposition

Every address is decomposed into three fields according to parameterized bit widths. The upper `PAGE_BITS` bits identify the *page*; the next `BLOCK_BITS` bits identify the *block* within the page; the remaining low-order bits identify the *byte* within the block. This three-level hierarchy mirrors the structure of a real memory subsystem and makes sparse allocation efficient even for large address spaces.

Security Model

The sparse memory supports read/write restrictions at three levels of granularity: page, block, and individual word. A restriction value of `RESTRICT_READ` prevents reads; `RESTRICT_WRITE` prevents writes; `RESTRICT_READ_WRITE` prevents both. Restrictions are consulted in order from coarsest (page) to finest (word); the first matching restriction wins. When a security violation occurs the offending operation is aborted and an error code is recorded.

Error Reporting

After every read or write operation the caller can inspect whether the operation succeeded. `last_operation_failed` returns `1` if an error occurred. `get_last_error()` returns the specific `error_t` value, and `get_last_op_string()` returns a human-readable string that names the operation, the address, and the reason for failure. This is particularly useful for debug and self-checking testbenches.

Bounded Memory

The `mem_bounded` class extends `mem` with configurable lower and upper address bounds. Any access outside those bounds is rejected with an `ERROR_ADDRESS_BOUNDS` error. An optional bounds lock prevents the bounds from being changed once set, which is useful for enforcing region ownership in a shared address space.

Implementation Notes

Internally the sparse memory is structured as a *map of pages*, where each page is a *map of blocks*, and each block is a *vector of bytes*. The maps use SVX's `map` class (a balanced tree), so lookup is $O(\log n)$ in the number of allocated pages or blocks. Only the `mem` and `mem_bounded` classes are intended for direct instantiation; `mem_page` and `mem_block` are internal implementation classes.

1.1 Parameters

All four classes share the same parameter set.

Parameter	Default	Description
ADDR_BITS	32	Total address space width in bits.
PAGE_BITS	16	Width of the page index field.
BLOCK_BITS	8	Width of the block index field within a page.
WORD_SIZE	4	Word size in bytes; words must be naturally aligned.

1.2 Types

Key typedefs (defined in `mem_types.svh` and `mem_base`):

Type	Description
addr_t	Parameterized address type: bit [ADDR_BITS-1:0].
word_t	Word type: bit [WORD_SIZE*8-1:0].
byte_t	8-bit unsigned byte.
pri_t	Priority type (uint32_t), used by pri_queue, not mem.
restrict_t	Security restriction enum (see below).
error_t	Error code enum (see below).
operation_t	Last operation enum (see below).

restrict_t

Value	Meaning
RESTRICT_NONE	No restriction; reads and writes are allowed.
RESTRICT_READ	Reads are blocked; writes are allowed.
RESTRICT_WRITE	Writes are blocked; reads are allowed.
RESTRICT_READ_WRITE	Both reads and writes are blocked.

error_t

Value	Meaning
ERROR_NONE	No error; the last operation succeeded.
ERROR_PARAMETERS_WRONG	Class parameters are inconsistent (fatal).
ERROR_MISALIGNMENT	Word access to a non-word-aligned address.
ERROR_PAGE_SECURITY_VIOLATION	A page-level restriction blocked the access.
ERROR_BLOCK_SECURITY_VIOLATION	A block-level restriction blocked the access.
ERROR_WORD_SECURITY_VIOLATION	A word-level restriction blocked the access.
ERROR_ADDRESS_BOUNDS	Address is outside the configured bounds (mem_bounded only).

operation_t

Value	Meaning
`OP_NONE`	No operation has been performed yet.
`OP_READ`	The last operation was a word read.
`OP_WRITE`	The last operation was a word write.
`OP_READ_BYTE`	The last operation was a byte read.
`OP_WRITE_BYTE`	The last operation was a byte write.

1.3 Class: `mem`

The top-level user-facing class. Declare and instantiate it directly:

```
mem#(ADDR_BITS, PAGE_BITS, BLOCK_BITS, WORD_SIZE) m = new();
```

Method	Returns	Arguments	Description
`write`	void	`addr_t addr` `word_t data`	Write a word to addr. Addr must be word-aligned.
`read`	`word_t`	`addr_t addr`	Read and return the word at addr. Returns 0 if the location was never written.
`write_byte`	void	`addr_t addr` `byte_t data`	Write a single byte to addr.
`read_byte`	`byte_t`	`addr_t addr`	Read and return the byte at addr. Returns 0 if never written.
`set_page_restriction`	void	`addr_t addr` `restrict_t r`	Apply restriction r to the entire page that contains addr.
`get_page_restriction`	`restrict_t`	`addr_t addr`	Return the restriction on the page containing addr.
`clear_page_restriction`	void	`addr_t addr`	Remove any page-level restriction for the page containing addr.
`set_block_restriction`	void	`addr_t addr` `restrict_t r`	Apply restriction r to the block containing addr.
`clear_block_restriction`	void	`addr_t addr`	Remove any block-level restriction for the block containing addr.
`set_word_restriction`	void	`addr_t addr` `restrict_t r`	Apply restriction r to the word at addr.
`clear_word_restriction`	void	`addr_t addr`	Remove any word-level restriction at addr.
`is_readable`	`bit`	`addr_t addr`	Return 1 if addr may be read (no active read restriction).
`is_writable`	`bit`	`addr_t addr`	Return 1 if addr may be written (no active write restriction).
`get_addr_restriction`	`restrict_t`	`addr_t addr`	Return the effective restriction at addr (coarsest level that applies).
`last_operation_failed`	`bit`	—	Return 1 if the most recent operation encountered an error.

Method	Returns	Arguments	Description
get_last_error	error_t	—	Return the error_t code from the most recent operation.
get_last_op_string	string	—	Return a human-readable string describing the last operation and any error.
dump_security	void	addr_t addr=0	Print the security map for all pages (and blocks if allocated).
dump	void	addr_t addr=0	Print the contents of all allocated pages and blocks.

1.4 Class: mem_bounded

mem_bounded extends mem with lower and upper address bounds. Any access outside the configured range is rejected with ERROR_ADDRESS_BOUNDS. All methods of mem are available; the following are added or overridden.

Method	Returns	Arguments	Description
set_bounds	void	addr_t lower addr_t upper	Set both the lower and upper bounds in one call.
set_lower_bound	void	addr_t lower	Set the lower bound only.
set_upper_bound	void	addr_t upper	Set the upper bound only.
get_lower_bound	addr_t	—	Return the current lower bound.
get_upper_bound	addr_t	—	Return the current upper bound.
set_bounds_lock	void	bit lock	Lock the address bounds when lock=1; further calls to set_bounds* are silently ignored.
get_bounds_lock	bit	—	Return 1 if the bounds are currently locked.

2. Memory Map

The `mem_map` application models a hierarchical address space of the kind found in hardware block specifications: registers, bit fields, sub-regions, memories, and overlapping views, all organized into a tree whose nodes carry precise offset and size information.

Overview

Each node in the hierarchy is a *memory space*, represented by the abstract base class `mem_space`. Concrete subclasses capture the five distinct space types recognized in hardware design: `MEMORY`, `REGISTER`, `FIELD`, `REGION`, and `VIEW`. Every space has a name, a parent, an offset (relative to the parent), and a size. The hierarchy is built by constructing child spaces and passing the parent as a constructor argument; the child registers itself with its parent automatically.

Space Types and Hierarchy Rules

The five space types form a strict containment hierarchy. Not every type may be nested inside every other type:

- A **FIELD** is a bit field within a register. Fields are leaves: they may not contain children.
- A **REGISTER** models a memory-mapped register. Its only legal children are FIELDs.
- A **MEMORY** represents a contiguous block of memory. Memories are leaves: they may not contain children.
- A **REGION** is a container for other spaces. It may contain REGISTERs, MEMORies, nested REGIONs, and VIEWs, but not FIELDs directly. Children of a region may not overlap each other.
- A **VIEW** is like a REGION, but sibling VIEWs *are* permitted to overlap each other. This models hardware designs where the same physical addresses can be interpreted in different ways depending on a mode bit.

Address Calculation and Validation

After the hierarchy is fully constructed, call `calculate_and_check()` on the root to propagate absolute addresses down the tree and to check for overlapping siblings. `get_addr()` and `get_end_addr()` then return the absolute address range of any node.

Navigation

`find_addr(addr)` searches the hierarchy for the deepest node whose range contains `addr` and returns it. `find_addr_all(addr)` returns a list of *all* nodes (at every level) whose range contains `addr`, which is useful when VIEWs overlap.

Implementation

`mem_space` extends SVX's `tree` class. Children are stored in an SVX `deque` and accessed through a typed iterator. The overlap check in `check()` iterates over all pairs of non-VIEW siblings and verifies that their address ranges do not intersect.

2.1 Space Types

Type	Leaf?	Allowed children
`FIELD`	Yes	None
`REGISTER`	No	FIELD only
`MEMORY`	Yes	None
`REGION`	No	REGISTER, MEMORY, REGION, VIEW
`VIEW`	No	REGISTER, MEMORY, REGION, VIEW (may overlap sibling VIEWs)

2.2 Class: `mem_space` (Abstract Base)

All concrete space classes extend `mem_space#(ADDR_SIZE=32)`. The methods below are defined on the base and inherited by all subtypes. Pure-virtual methods (`get_type()`, `check_child()`) are implemented by each concrete subclass.

Method	Returns	Arguments	Description
`get_offset`	`addr_t`	—	Return the offset of this space relative to its parent.
`get_size`	`mem_size_t`	—	Return the size of this space in address units.
`get_addr`	`addr_t`	—	Return the absolute base address (valid after calculate_and_check()).
`get_end_addr`	`addr_t`	—	Return the absolute end address (base + size - 1).
`get_type`	`mem_space_type_t`	—	Return the space type (FIELD, REGISTER, MEMORY, REGION, or VIEW). Pure virtual.
`get_type_name`	`string`	—	Return the space type as a human-readable string.
`check_child`	`bit`	`mem_space_t child`	Return 1 if child is a legal child of this space type. Pure virtual.
`insert_space`	void	`mem_space_t child`	Insert child into this space; emits a warning and skips insertion on type violation.
`get_error`	`bit`	—	Return 1 if this space has been flagged as erroneous.

Method	Returns	Arguments	Description
`clear_error`	void	—	Clear the stored error message.
`calculate_and_check`	`bit`	—	Propagate absolute addresses and check for overlapping siblings. Call on the root after the hierarchy is fully built.
`find_addr`	`mem_space_t`	`addr_t addr`	Find and return the deepest space whose range contains addr.
`find_addr_all`	`list_t`	`addr_t addr`	Return a list of all spaces (at every level) whose range contains addr.
`dump`	void	—	Print the hierarchy rooted at this node, with offsets, sizes, and absolute addresses.

2.3 Classes: `mem_region` and `mem_view`

`mem_region#(ADDR_SIZE=32)` and `mem_view#(ADDR_SIZE=32)` both extend `mem_space` and provide factory methods for building the hierarchy. `mem_view` additionally permits sibling instances to overlap in address space. Constructor: `new(name, parent, offset, size)`.

Method	Returns	Arguments	Description

`add_register`	void	`string name`
		`addr_t offset`
		`mem_size_t size`
			Create and attach a mem_register child at the given offset and size.

`add_memory`	void	`string name`
		`addr_t offset`
		`mem_size_t size`
			Create and attach a mem_memory child at the given offset and size.

`add_region`	void	`string name`
		`addr_t offset`
		`mem_size_t size`
			Create and attach a nested mem_region child.

`add_view`	void	`string name`
		`addr_t offset`
		`mem_size_t size`
			Create and attach a mem_view child.

2.4 Class: `mem_register`

`mem_register#(ADDR_SIZE=32)` extends `mem_space`. Its only legal children are FIELDs. Constructor: `new(name, parent, offset, size)`.

Method	Returns	Arguments	Description

`add_field`	void	`string name`
		`addr_t offset`
		`mem_size_t size`
			Create and attach a mem_field child representing a bit field at the given bit offset and width.

2.5 Classes: `mem_memory` and `mem_field`

Both are leaf nodes with no children and no additional methods beyond those inherited from `mem_space`. They exist primarily as type tags so that `check_child()` can enforce the containment rules. Constructor for both: `new(name, parent, offset, size)`.

3. Priority Queue

The `pri_queue` application is a generic priority queue that supports two fundamental operations: `push()` and `pop()`.

Overview

`push(pri, item)` inserts `item` into the queue at priority level `pri`. `pop()` removes and returns the item with the highest priority. When multiple items share the same priority they are returned in the order they were inserted — first-in, first-out within each priority level.

Implementation

The underlying data structure is a *map of queues*: an SVX `map` whose keys are priority values and whose values are SVX `queue` instances holding the items at each priority. Because SVX's `map` is implemented as an ordered tree, the entry with the highest key (priority) is retrieved in constant time via `last()`. When the last item at a given priority is popped, the corresponding map entry is deleted automatically, keeping the map compact.

Type Parameters

`pri_queue` is doubly parameterized: `T` is the item type (default `int`) and `P` is the traits class (default `void_traits`). The priority type `pri_t` is a `uint32_t` defined in `pri_queue_pkg`. `pri_queue` extends SVX's `typed_container#(T,P)` and inherits its `is_empty()`, `size()`, and `clear()` methods.

3.1 Class: `pri_queue`

Declared as `pri_queue#(type T=int, type P=void_traits)` in `pri_queue_pkg`. Constructor: `new()` — no arguments required.

Method	Returns	Arguments	Description
`push`	void	`pri_t pri` `T item`	Insert item into the queue at priority level pri. Higher values of pri represent higher priority.
`pop`	`T`	—	Remove and return the item with the highest priority. When multiple items share the top priority, returns the one inserted earliest. Returns P::empty if the queue is empty.
`is_empty`	`bit`	—	Return 1 if the queue contains no items.
`size`	`size_t`	—	Return the total number of items across all priority levels.
`clear`	void	—	Remove all items from the queue.